

Module 00 — Orientation: The Map

FOR DUMMIES

This repo has a lot in it. That's deliberate — it's a toolbox, not a single tool — but toolboxes are confusing until someone tells you which drawer holds what. This module is that.

Read it once now (10 minutes), then come back to it whenever you're mid-lab thinking *"which file do I even edit for this?"*

1. The two planes (the single most important idea)

Everything in this repo belongs to one of two planes, and confusing them is the #1 way people get lost:

TOPOLOGY PLANE — "what does the network LOOK like?"

Which devices exist, what image they run, how they're cabled.

Owned by: `topologies/*.clab.yml`

Changed by: `edit YAML -> containerlab destroy -> deploy`

Think: racking hardware and running cables

CONFIG PLANE — "what do the devices SAY?"

Interface IPs, OSPF, BGP, VLANs, VXLAN — the running-config.

Owned by: `automation/nornir/templates/*.j2` + data in scripts

Changed by: `edit template/data -> run the script (no redeploy)`

Think: SSHing in and typing configure terminal

The workflows are completely different:

	Topology change	Config change
Example	"add a third leaf"	"change leaf1's loopback IP"
Edit	<code>topologies/spine-leaf.clab.yml</code>	template or data dict in <code>automation/nornir/</code>
Apply	destroy then deploy (containers are replaced)	run the Nornir script (devices stay up)
Disruptive?	Yes — nodes reboot	No — NAPALM merges the diff

	Topology change	Config change
Then	re-run <code>link-inventory.sh</code> (mgmt IPs changed)	verify with show commands

A topology change invalidates the inventory (new containers, new management IPs), which is why `link-inventory.sh` exists and why it runs after every deploy. A config change touches nothing but the devices.

2. The file map

What every file is, one line each, grouped by when you touch it.

You run these (entry points)

File	What it does
<code>setup.sh</code>	Guided interactive setup — the “just make it work” path. Run once.
<code>./lab</code>	The day-to-day driver: <code>status / deploy / ssh / cli / graph / destroy / topo</code> . Run it bare for a menu. This is the one to remember.
<code>scripts/00-bootstrap.sh</code>	Installs Docker, Containerlab, Python env. Idempotent.
<code>scripts/01-import-ceos.sh</code>	Verifies the cEOS tarball's hash, imports into Docker
<code>scripts/02-deploy-lab.sh</code>	Deploy / destroy / inspect / graph a topology. Checks images exist locally first.
<code>scripts/03-build-vrnetlab.sh</code>	Builds VM-based node images (FortiGate) from a vendor <code>.qcow2</code> via <code>vrnetlab</code> . Needs nested virt.
<code>scripts/sync-labs.sh</code>	Copies/updates <code>~/labs/topologies</code> from the repo, leaving files you've edited alone
<code>automation/link-inventory.sh</code>	Points Nornir & Ansible at the <i>current</i> lab's inventory

You edit these (topology plane)

File	What it does
<code>topologies/testlab.clab.yml</code>	2-node smoke test. Don't grow this one — copy it.

File	What it does
topologies/spine-leaf.clab.yml	The course fabric: 2 spines, 2 leaves
topologies/memory-test.clab.yml	Bigger mixed lab: 2x FortiGate + 4x cEOS + FRR "ISPs" + hosts. Needs the vrnetlab image built first.
(yours) topologies/<name>.clab.yml	Copy an existing one and go

You edit these (config plane)

File	What it does
automation/nornir/templates/*.j2	Jinja2 config templates — the <i>shape</i> of device config
FABRIC / DEMO_DATA dicts in the 0*.py scripts	Per-device values the templates consume — the <i>data</i>
automation/nornir/0*_*.py	The scripts that render + push. Numbered = course order.

You mostly leave these alone (plumbing)

File	What it does
automation/nornir/config.yml	Tells Nornir where the inventory is. Set once.
automation/nornir/nornir-simple-inventory.yml	A symlink. Containerlab regenerates the real file every deploy; never edit it.
automation/nornir/creds.py	Reads NORNIR_USERNAME/PASSWORD env vars if set, else no-op
automation/pyproject.toml	Python deps (uv's source of truth)
automation/requirements.txt	Same deps, for the pip fallback
lib/ui.sh	The gum/plain-bash UI helpers setup.sh and ./lab use

Two conventions worth knowing

Image tags. Topologies reference `ceos:latest`, not a pinned version — everyone downloads a different cEOS build from Arista, so `01-import-ceos.sh` tags whatever you import as both `ceos:<version>` and `ceos:latest`. `docker images` still shows the real version. Get the images from the [team Drive](#) or the vendor portals.

Two copies of every topology. The repo has `topologies/`, and `~/labs/topologies/` is the working copy `02-deploy-lab.sh` actually deploys from. `sync-labs.sh` keeps them in step — it updates copies it installed and leaves ones you’ve edited alone. If a change to a topology seems to have no effect, this is why.

Optional drawers (open when the course says to)

File	What it does
<code>automation/nornir/SECRETS.md + run-with-*.sh + secrets.example.yaml + .env.example</code>	Module 06: three credential backends for the Nornir path
<code>automation/ansible/**</code>	The whole Ansible alternative: <code>ansible.cfg</code> , playbooks, <code>group_vars/ceos/</code> (incl. Vault)
<code>automation/lib/vault.py</code>	Lets Python scripts read the Ansible Vault file too
<code>docs/vrnetlab-fortigate.md</code>	Building FortiGate (and other VM-based nodes) with <code>vrnetlab</code> , incl. the nested-virt hypervisor setting
<code>docs/*.md</code>	VM builds (vCenter / Proxmox) and how to distribute this repo

Generated at runtime (never in git, never hand-edited)

Path	What it is
<code>clab-<labname>/</code> (next to the topology file)	Containerlab’s working dir: node startup-configs, TLS certs, the generated inventories
<code>~/venvs/netauto/</code>	The Python environment (<code>netauto</code> alias activates it)
<code>/tmp/clab-setup.log</code>	Everything <code>setup.sh</code> did, when you need to know why it failed
<code>~/labs/topologies/</code>	The working copies you actually deploy; <code>sync-labs.sh</code> keeps them current
<code>~/vrnetlab/</code>	<code>vrnetlab</code> clone, if you’ve built a VM-based node image

3. The decision tree

Start at the top, follow your situation down.

```
"I want to..."
|
├─ ...set up a fresh VM / fix a broken install
|   └─> ./setup.sh           (re-running is safe; done steps are skipped)
|
```

```

└─ ...change WHAT DEVICES EXIST or HOW THEY'RE CABLED
    └─ edit topologies/<lab>.clab.yml
        ./scripts/02-deploy-lab.sh topologies/<lab>.clab.yml destroy
        ./scripts/02-deploy-lab.sh topologies/<lab>.clab.yml
        ./automation/link-inventory.sh <path-to-clab-<labname>>
            (mgmt IPs changed -> inventory must be re-linked)

└─ ...change WHAT THE DEVICES ARE CONFIGURED WITH
    │
    └─ one-off / exploring? -> ssh admin@clab-<lab>-<node>, configure by hand
        (fine for learning; it's what module 03 does)
    │
    └─ repeatable / at scale? -> edit the .j2 template (config SHAPE changed)
        and/or the data dict (VALUES changed)
        then: python3 0X_<script>.py
        run twice - 2nd run must say changed=False

└─ ...just LOOK at the running lab
    │
    └─ table of nodes+IPs: ./scripts/02-deploy-lab.sh <topo> inspect
    │
    └─ visual map: ./scripts/02-deploy-lab.sh <topo> graph
        then browse to http://<vm-ip>:50080
    │
    └─ resource usage: docker stats
    └─ a node's boot logs: docker logs clab-<lab>-<node>

└─ ...get INTO a device
    │
    └─ normal: ssh admin@clab-<lab>-<node> (pw: admin)
    └─ SSH broken/locked out: docker exec -it clab-<lab>-<node> Cli

└─ ...stop for the day
    └─ on each node: write memory
        ./scripts/02-deploy-lab.sh <topo> destroy
        (configs survive in clab-<labname>/ - next deploy restores them)

└─ ...start FRESH, discarding all saved configs
    └─ destroy, then delete the clab-<labname>/ directory, then deploy

└─ ...stop using default admin/admin credentials
    └─ Module 06 / automation/nornir/SECRETS.md

```

4. Checkpoint

You're done with this module when you can answer these without looking:

1. You want to add a third leaf. Which plane is that, which file do you edit, and what two commands follow — plus the one people forget?
2. You want to change the OSPF process ID everywhere. Template, data, or topology file?
3. Why must you never hand-edit `nornir-simple-inventory.yml`?

► Answers

Next: [Module 01 — The platform](#)